Nexus

NEXUS— Final Backend PRD with Build Phases

Version 2.0 | Offline Mesh Communication System

DECISION RESOLUTIONS

PWA only. No Node server. This changes the hotspot sync architecture significantly and needs to be stated upfront clearly.

A pure PWA cannot open a TCP server socket. The browser has no server API. This means Device B cannot passively serve data to Device C. Instead, the sync model flips: both devices must actively participate. The joining device cannot simply hit an IP and pull data. The practical solution is WebRTC data channels with QR-based signaling, which is fully offline, fully browser-native, and requires no server.

Here is how it works. Device B generates an "offer" — a WebRTC session description — and displays it as a QR code. Device C scans that QR, generates an "answer" session description, and displays it as a QR. Device B scans Device C's answer QR. After two QR scans, a live WebRTC data channel exists between B and C over the local Wi-Fi network with no server involved. Data flows peer-to-peer through that channel. This is the hotspot sync mechanism.

For the demo, the two-QR handshake takes about 30 seconds total. It is slightly more friction than a URL-based approach but it is 100% offline, 100% browser-native, and actually more impressive to judges because it demonstrates the system solving the server problem creatively.

Audio capture platform decision. Use the Web Speech Recognition API with a graceful fallback. On iOS 14.5+ it runs on-device and works offline. On Android, test your specific device before the hackathon — some Android builds have on-device recognition, many require network. If the demo device fails offline recognition, the fallback is manual text entry with the capture UI still shown. The user hears the broadcast and types what they heard. Not ideal but functional. Label it in the UI as "Manual Transcription" mode. The audio broadcast (outbound) works on all devices regardless since Speech Synthesis is universally on-device.

Bloom filter. Built only when a WebRTC data channel is active. The channel gives you a low-bandwidth bidirectional connection, which is exactly the right place for a bloom filter pre-exchange. QR transfers skip bloom filter entirely — too slow for the handshake. Implementation is in Phase 4.

Physical QR drop point. Skipped for hackathon as decided.

SYSTEM ARCHITECTURE REVISED FOR PWA

Each device runs a PWA with a Service Worker handling offline caching of the app shell. All data lives in IndexedDB. Communication happens through three channels:

Channel 1 — Snapshot QR: one device encodes messages into a QR image, the other scans it. Asynchronous, no live connection needed, works across time (physical drop point use case).

Channel 2 — WebRTC via QR signaling: two devices exchange WebRTC offer/answer via QR codes, establish a live data channel over local Wi-Fi or any shared network. Synchronous, bidirectional, enables continuous polling and bloom filter exchange. This replaces the hotspot HTTP model.

Channel 3 — Audio broadcast and capture: one device plays a message via Speech Synthesis, nearby device captures via Speech Recognition and ingests the transcript.

The Prioritization-Routing Agent, Sentinel, Ingestion function, Privacy function, Merge function, and QR Encoding function all remain exactly as specified in the previous PRD. The only architectural change is replacing the Node HTTP server and its endpoints with a WebRTC data channel and a message-passing protocol over that channel.

WEBRTC DATA CHANNEL PROTOCOL

This replaces all HTTP endpoints from the previous PRD. The data channel is established once via QR signaling, then both peers communicate by sending JSON strings over the channel.

Message types over the data channel:

BLOOM_OFFER — sender transmits their bloom filter. Receiver uses it to filter outbound messages.

```
{ type: "BLOOM_OFFER", bloom: "base64string", message_count: integer }
```

BLOOM_ACK — receiver acknowledges bloom filter received.

```
{ type: "BLOOM_ACK" }
```

MSG_PUSH — sender pushes a single message to receiver. One message per frame. Receiver runs full ingestion on it.

```
{ type: "MSG_PUSH", message: { ...full message object } }
```

MSG_ACK — receiver acknowledges a message. Includes status.

```
{ type: "MSG_ACK", id: "string", status: "accepted" | "duplicate" | "invalid" | "expired" }
```

SYNC_DONE — sender signals it has no more messages to push in this direction.

```
{ type: "SYNC_DONE", sent_count: integer }
```

STATUS — either peer can request the other's storage status at any time.

```
{ type: "STATUS_REQ" }
{ type: "STATUS_RES", message_count: integer, storage_used_bytes: integer, schema_version: 1
}
```

Sync session flow over WebRTC data channel:

1. Channel opens. Both peers simultaneously send STATUS_REQ.
2. Both peers receive STATUS_RES. If schema_version does not match, both peers send SYNC_DONE immediately and close. Incompatible versions do not exchange data.
3. Both peers simultaneously send BLOOM_OFFER if bloom filter is implemented (Phase 4). Otherwise skip to step 5.
4. Both peers send BLOOM_ACK on receipt.
5. Both peers simultaneously begin pushing messages. Prioritization Agent runs, selects top-N messages not in peer's bloom filter (or full top-N if no bloom filter), pushes them one MSG_PUSH at a time.
6. Each MSG_PUSH waits for MSG_ACK before sending the next. This keeps the channel from flooding and makes each transfer atomic.
7. When a peer has no more messages to push, it sends SYNC_DONE.

8. When both peers have sent SYNC_DONE, the session ends. Either peer may close the channel.
9. Both peers write session summary to encounter_log.

WebRTC signaling via QR:

The WebRTC offer and answer are SDP strings — typically 1-2KB. This fits in a single QR at Medium error correction after compression.

Initiator (Device B, the one who wants to share):

1. Create RTCPeerConnection.
2. Create data channel.
3. Generate offer via createOffer().
4. Set local description.
5. Compress the SDP string with pako.
6. Base64 encode.
7. Prepend "W:" prefix (WebRTC mode flag, distinct from QR snapshot "S:" and live "L:").
8. Display as QR code.

Joiner (Device C, the one who wants to connect):

1. Scan Device B's QR.
2. Detect "W:" prefix.
3. Decompress SDP.
4. Create RTCPeerConnection.
5. Set remote description with the offer.
6. Generate answer via createAnswer().
7. Set local description.
8. Compress answer SDP.
9. Base64 encode.

10. Prepend "W:" prefix.
11. Display as QR code.

Initiator completes:

1. Scan Device C's answer QR.
2. Set remote description with the answer.
3. ICE candidates exchange automatically over the local network.
4. Data channel opens. Sync session begins.

Total handshake: two QR scans, approximately 30 seconds. After that, the channel is live and sync runs continuously until either device closes it.

FINAL MESSAGE SCHEMA

Identical to previous PRD. Restated here for completeness as the single source of truth.

```
{
  id:             string   — first 16 chars of SHA-256(type + created_at + payload)
  type:           string   — "text" | "audio" | "alert"
  priority:       integer  — 1 to 5, set at creation, immutable
  ttl:            integer  — Unix timestamp (seconds), hour-resolution, message self-
destructs after this
  created_at:     integer  — Unix timestamp (seconds), hour-resolution, set at creation,
never modified
  hop_count:      integer  — 0 at creation, incremented by 1 on every ingest at a new device
  weight:         float    — 1.0 at creation, +0.15 per relay, max 10.0
  payload:        string   — UTF-8 text for all types; audio type contains transcribed text
  confidence:     string   — "high" for text and alert; "low" for audio
  supersedes:     string   — optional, ID of message this declares itself an update to
  schema_version: integer  — always 1
}
```

Computed at read time, never stored:

```
temperature:  "hot" | "warm" | "cold"
score:        float
is_expired:   boolean
is_superseded: boolean
```

Temperature derivation: hot if weight >= 6.0 AND hours_since_created <= 12. Cold if weight < 3.0 AND hours_since_created > 48. Warm otherwise. Alert type always hot regardless.

Default TTLs: alert = 6 hours, text = 72 hours, audio = 48 hours.

BUILD PHASES

PHASE 1 — FOUNDATION

Goal: IndexedDB working, messages can be created and read, basic UI renders a message list. Nothing transfers yet. This is the data layer.

What to build:

IndexedDB setup. Create database "bridge_db" with three object stores: messages (keyPath: "id"), encounter_log (keyPath: "session_id"), system_state (keyPath: "key"). Create indexes on messages store: by_ttl on ttl field, by_weight on weight field, by_created_at on created_at field, by_type on type field.

Message creation function. Takes type, priority, payload, optional ttl_hours. Computes id as SHA-256 of (type + created_at_floored + payload), takes first 16 hex chars. Sets created_at to current time floored to nearest hour. Sets ttl to created_at plus ttl_hours converted to seconds, floored to nearest hour. Sets hop_count to 0, weight to 1.0, confidence based on type, schema_version to 1. Writes to messages store. Returns the complete message object.

Message read functions. getAll() returns all non-expired messages sorted by score descending. getById(id) returns single message or null. getByType(type) returns filtered list. getByTemperature(temp) returns filtered list.

Computed field derivation. A pure function computeFields(message) that takes a raw message record and returns it with temperature, score, is_expired, is_superseded added. This function is called on every read, never on write. Score formula: (priority × 2) × (weight / 10) × (1 / (1 + hours_since_created × 0.1)) × temperature_multiplier / (1 + hop_count × 0.2). Temperature multiplier: hot = 3.0, warm = 1.5, cold = 0.5.

Expiry sweep. A function sweepExpired() that deletes all messages where ttl < current Unix timestamp. Runs every 60 seconds via setInterval, and on every write.

Eviction policy. A function runEviction() that checks storage usage via navigator.storage.estimate(). If usage exceeds 45MB, sorts all non-alert non-hot

messages by weight ascending and deletes from lowest until usage drops below 35MB. Never deletes alerts. Never deletes hot messages.

Service Worker. Caches the app shell (HTML, CSS, JS, libraries) on install. Serves from cache on fetch. No network requests to cache — the app has no network to speak to. This ensures the PWA loads even when the phone has been restarted with no connectivity.

Libraries to install in Phase 1:
idb (IndexedDB wrapper, ~5KB), pako (compression), qrcode (QR generation), jsQR (QR decoding from image data).

Phase 1 tests:

Test 1.1 — Create a text message with priority 3. Verify id is 16 hex characters. Verify created_at is floored to nearest hour. Verify hop_count is 0 and weight is 1.0.

Test 1.2 — Create two messages with identical type, payload, and created_at. Verify they produce the same id. Verify that writing the second one is a no-op (deduplication at write time).

Test 1.3 — Create a message with ttl set to 1 second in the future. Wait 2 seconds. Run sweepExpired(). Verify message is no longer in storage.

Test 1.4 — Create 10 messages with varying priority, weight, and created_at values. Call getAll(). Verify they are returned in score-descending order. Manually verify the top result is the one you would expect given the formula.

Test 1.5 — Create a message with type "alert" and priority 2. Verify computeFields() overrides temperature to "hot" regardless of weight and age.

Test 1.6 — Create a message with weight 1.0 and created_at 3 days ago. Verify temperature is "cold". Create a message with weight 7.0 and created_at 6 hours ago. Verify temperature is "hot".

Test 1.7 — Simulate storage pressure by mocking navigator.storage.estimate() to return 48MB. Create 20 messages with varying weights including 3 alerts and 2 hot-temperature messages. Run runEviction(). Verify no alerts were deleted. Verify no hot messages were deleted. Verify lowest-weight warm/cold messages were deleted first.

Test 1.8 — Close the browser tab. Reopen the PWA. Verify all messages are still present (IndexedDB persistence check).

Test 1.9 — Load the app with the device in airplane mode. Verify the Service Worker serves the app shell and the app renders. Verify no network errors in console.

PHASE 2 — INGESTION, PRIVACY, MERGE, QR

Goal: messages can be created, validated, ingested from external sources, and transferred via QR. End of this phase, two devices can exchange data through QR scan.

What to build:

Ingestion function. Takes a raw incoming object. Runs in this exact order: check schema_version (quarantine if not 1), validate all required fields present (discard if missing), validate field types and ranges (discard if invalid), check id exists in storage (discard if duplicate), check ttl against current time (discard if expired), run Privacy function, increment hop_count by 1, increment weight by 0.15 capped at 10.0, write to storage, trigger scoring loop, return status string "accepted" | "duplicate" | "invalid" | "expired" | "quarantined".

Privacy function. Runs on every message before storage and before any outbound relay. Strip any fields not in schema. Strip sender field if present. Strip device_id field if present. Floor created_at to nearest hour. Floor ttl to nearest hour.

Merge function. Runs inside Ingestion after expiry check, only when incoming message has a supersedes field. Looks up the superseded ID. If found, adds superseded_by field to the old record pointing to new message ID. Stores updated old record. Proceeds with normal ingestion of new message.

Prioritization Agent scoring loop. A function runScoringLoop() that reads all messages from storage, calls computeFields() on each, and updates the score field in storage. Does not rewrite the full record — only updates the score and temperature fields. Runs via setInterval every 15 seconds. Also triggered immediately by Ingestion on any successful write.

QR Encoding function — Snapshot mode. Takes top-N messages from Prioritization Agent output where N starts at 5 and decrements until the compressed payload fits in 2.3KB. Serializes as JSON array. Compresses with

pako.deflate(). Base64 encodes. Prepends "S:". Passes to qrcode library. Returns base64 PNG data URL of QR image.

QR Encoding function — WebRTC offer mode. Takes a compressed SDP string, base64 encodes, prepends "W:". Passes to qrcode library. Returns base64 PNG data URL.

QR Decoding function. Takes image data from camera (ImageData object from canvas). Passes to jsQR. If jsQR returns null, return null. Read first two characters. If "S:", strip prefix, base64 decode, pako.inflate(), JSON.parse(), run Ingestion on each message in array, return array of ingest results. If "W:", strip prefix, base64 decode, pako.inflate(), return SDP string for WebRTC handler. If "L:", strip prefix, return URL string (legacy live QR, not generated in this version but should be handled gracefully). Any other prefix, return error "unknown_qr_type".

Camera scanner. Opens getUserMedia with video constraint facing environment (rear camera). Draws frames to a hidden canvas at 200ms intervals. Extracts ImageData and passes to QR Decoding function. Stops scanning on successful decode or user cancellation.

Phase 2 tests:

Test 2.1 — Create a valid message object manually. Run Ingestion. Verify status is "accepted". Verify hop_count is 1 (was 0 on input). Verify weight is 1.15 (was 1.0 on input). Verify Privacy function stripped any extra fields.

Test 2.2 — Run Ingestion on the same message again. Verify status is "duplicate". Verify message count in storage has not changed.

Test 2.3 — Run Ingestion on a message missing the payload field. Verify status is "invalid". Verify nothing written to storage.

Test 2.4 — Run Ingestion on a message with ttl set to 1 hour ago. Verify status is "expired". Verify nothing written to storage.

Test 2.5 — Run Ingestion on a message with schema_version set to 99. Verify status is "quarantined". Verify the raw string appears in system_state quarantine key. Verify nothing in messages store.

Test 2.6 — Create a message A. Create a message B with supersedes field set to A's id. Run Ingestion on B. Verify A's record in storage now has superseded_by field pointing to B's id.

Test 2.7 — Create a message with a sender field set to "device_xyz". Run Ingestion. Read the stored message. Verify no sender field exists.

Test 2.8 — Create a message with created_at set to a precise timestamp (e.g., 14:37:22). Run Ingestion. Verify stored created_at is 14:00:00 (floored to hour).

Test 2.9 — Call generateSnapshotQR() with 3 messages in storage. Verify a base64 PNG is returned. Verify the QR can be decoded by the decoding function. Verify decoded content matches the original messages after decompression.

Test 2.10 — Create a snapshot QR with 5 messages where the compressed payload would exceed 2.3KB. Verify the function decrements N and retries, ultimately producing a valid QR with fewer messages.

Test 2.11 — Physical two-device test. On Device A, create 3 messages. Generate Snapshot QR. On Device B, open the scanner. Scan Device A's QR. Verify all 3 messages appear in Device B's storage with hop_count 1.

Test 2.12 — Run the scoring loop with 10 messages. Verify scores are updated in storage. Verify messages returned by getAll() are in score-descending order after the loop runs.

PHASE 3 — WEBRTC SYNC AND SENTINEL

Goal: two devices can establish a live data channel via QR handshake and sync bidirectionally. Sentinel is operational. End of this phase, the full demo flow works.

What to build:

WebRTC peer connection setup. Create RTCPeerConnection with ICE configuration set to empty (no STUN, no TURN — purely local network). On local network, ICE candidates are discovered via host candidate type only. No STUN server is needed or available.

Initiator flow. createPeerConnection(), createDataChannel("bridge_sync"), createOffer(), setLocalDescription(), wait for ICE gathering to complete (iceGatheringState === "complete"), compress the full local description SDP, generate WebRTC QR, display to user with instruction "Ask the other device to scan this."

Joiner flow. Scan initiator's "W:" QR. Decompress SDP. createPeerConnection(), setRemoteDescription(offer), createAnswer(), setLocalDescription(), wait for ICE

gathering complete, compress answer SDP, generate WebRTC QR, display to user with instruction "Ask the first device to scan this."

Initiator completion. Scan joiner's answer QR. setRemoteDescription(answer). Data channel opens. ondataopen fires.

Data channel sync session. On channel open, both peers run the sync session flow: send STATUS_REQ, receive STATUS_RES (check schema_version compatibility), then begin pushing messages via MSG_PUSH one at a time, waiting for MSG_ACK before sending next, ending with SYNC_DONE when queue is empty. Both directions run simultaneously — each peer is pushing and receiving at the same time.

Encounter log write. On session close, write a record to encounter_log with session_id (random 8-char hex), approximate_time (current hour), message_ids_exchanged (union of all ids sent and received in this session).

Sentinel — triple shake detection. Add DeviceMotionEvent listener on app load. Track acceleration magnitude: sqrt($x^2 + y^2 + z^2$) for each event. If magnitude exceeds 25, record timestamp. If three such events occur within a 2-second window, trigger Level 3 wipe.

Sentinel — iOS permission. On first app load, call DeviceMotionEvent.requestPermission() if the function exists (iOS 13+). If permission is denied, show a UI warning that the emergency wipe feature is unavailable and suggest using the manual wipe button in settings instead.

Sentinel — Level 2 selective wipe. Triggered by 3 wrong PIN attempts. Delete all messages where type is "alert". Delete all messages where computeFields() returns temperature "hot". Leave all other messages. Reload app.

Sentinel — Level 3 full wipe. Triggered by triple shake. Execute in sequence: clear messages store, clear encounter_log store, clear system_state store, clear localStorage, clear sessionStorage, clear all Cache API entries, call indexedDB.deleteDatabase("bridge_db"), reload page. Target completion under 500ms.

Sentinel — optional PIN setup. In settings, user can set a 6-digit numeric PIN. Store as bcrypt hash (cost factor 10) in system_state under key "pin_hash". On app open, if pin_hash exists, show PIN entry screen before rendering app. On 3

wrong attempts, trigger Level 2 wipe. No PIN recovery mechanism — if forgotten, user must do a manual wipe from settings.

Manual wipe option. In settings, a clearly labeled "Emergency Wipe" button that triggers the same Level 3 wipe as triple shake. Requires a single tap confirmation dialog — the only place in the sentinel system where a confirmation exists.

Phase 3 tests:

Test 3.1 — Two devices on the same Wi-Fi network. Device A generates WebRTC offer QR. Device B scans it, generates answer QR. Device A scans answer. Verify data channel opens on both devices (both show "Connected" state).

Test 3.2 — After channel opens, verify STATUS_REQ and STATUS_RES are exchanged and schema_version is checked. Disconnect and verify encounter_log has a new record.

Test 3.3 — Device A has 5 messages. Device B has 2 different messages. After full sync session (both sides send SYNC_DONE), verify Device A has 7 messages and Device B has 7 messages.

Test 3.4 — During sync, send a message that Device B already has. Verify Device B's Ingestion returns "duplicate" and the MSG_ACK carries status "duplicate". Verify Device B's message count did not change.

Test 3.5 — Full A→B→C demo test. Device A creates a message. A and B sync via WebRTC. Verify message on B with hop_count 1. B and C sync via WebRTC. Verify message on C with hop_count 2. Verify A and C never directly connected.

Test 3.6 — Shake simulation. Trigger three rapid shake events programmatically by firing DeviceMotionEvent with magnitude 30 three times within 1 second. Verify Level 3 wipe executes. Verify messages store is empty after wipe. Verify app reloads.

Test 3.7 — Set a PIN. Lock the app by simulating re-open. Enter wrong PIN three times. Verify Level 2 wipe runs: alerts and hot messages deleted, warm and cold messages remain.

Test 3.8 — Set a PIN. Lock app. Enter correct PIN. Verify app unlocks without wiping.

Test 3.9 — Trigger wipe while a WebRTC sync session is active. Verify wipe completes cleanly and the data channel is closed without sending any more data.

Test 3.10 — Wipe timing test. With 200 messages in storage, trigger Level 3 wipe and measure time from trigger to page reload. Verify it completes in under 500ms on a mid-range test device.

PHASE 4 — BLOOM FILTER

Goal: when a WebRTC data channel is active, exchange bloom filters before sync to avoid sending messages the peer already has. Reduces redundant data over low-bandwidth connections.

What to build:

Bloom filter implementation. A bloom filter with the following parameters: bit array size 16384 bits (2KB), 4 hash functions, expected false positive rate approximately 1% at 1000 messages. Use a simple implementation with no external library — the core is a Uint8Array of 2048 bytes and 4 hash functions derived from the message id string. The filter supports two operations: add(id) to set bits, and test(id) to check if bits are set. A test returning true means the peer probably has the message (but may be a false positive). A test returning false means the peer definitely does not have the message.

Bloom filter build function. Takes all message ids from storage, runs add() on each, returns the populated filter as a Uint8Array.

Bloom filter serialization. Uint8Array → base64 string for transmission over the data channel.

Integration with sync session. After STATUS_RES exchange and before MSG_PUSH flow: both peers send BLOOM_OFFER with their serialized bloom filter. Both peers receive BLOOM_ACK. Prioritization Agent's encounter-time selection now filters candidates through the peer's bloom filter using test(). Only messages where test() returns false are pushed. Messages where test() returns true are assumed to already be present and skipped.

False positive handling. If a MSG_PUSH is sent for a message the peer actually has (false positive from bloom filter), the peer's Ingestion returns "duplicate" and the MSG_ACK carries status "duplicate". The sender notes this but does not penalize the bloom filter — false positives are expected and the cost is one wasted message push.

Phase 4 tests:

Test 4.1 — Build a bloom filter from 100 message ids. Serialize and deserialize it. Verify that all 100 original ids test as true after deserialization (no false negatives — this is a hard guarantee of bloom filters).

Test 4.2 — Test 500 random strings that were not added to the filter. Verify false positive rate is below 5% (should be approximately 1% but allow margin for test variability).

Test 4.3 — Run a full WebRTC sync session between two devices where Device A has 10 messages and Device B already has 7 of them. Verify only 3 MSG_PUSH messages are sent from A to B (bloom filter correctly identified the 7 already-present). Allow 1 false positive, so accept 3 or 4 pushes as a pass.

Test 4.4 — Device A and Device B have completely disjoint message sets (0 in common). Verify all messages are exchanged in both directions. Bloom filter should not filter anything out.

Test 4.5 — Device A and Device B have identical message sets (100% overlap). Verify 0 MSG_PUSH messages are sent in either direction (all filtered by bloom). Allow up to 2 false negatives (which would result in 2 pushes that return "duplicate").

Test 4.6 — Verify that a SYNC_DONE is still sent correctly when the bloom filter filters out all candidates on one or both sides.

PHASE 5 — AUDIO BROADCAST AND CAPTURE

Goal: messages can be broadcast as audio via Speech Synthesis, and captured from audio via Speech Recognition into the message database.

What to build:

Audio broadcast. A function broadcastMessage(message). Calls window.speechSynthesis.cancel() to clear any existing queue. Creates a SpeechSynthesisUtterance with text set to message.payload. Sets rate to 0.9, volume to 1.0, lang to "en-IN" (Indian English, appropriate for target deployment context). Calls window.speechSynthesis.speak(). Returns a promise that resolves when speech ends (onend event) or rejects on error (onerror event).

Alert broadcast behavior. If type is "alert", prepend "URGENT ALERT: " to the spoken text. No other behavioral difference.

Broadcast queue. If user triggers broadcast while another is playing, queue the new message. Alerts jump to front of queue regardless of when they were queued.

Audio capture — standard mode. A function startCapture(). Creates SpeechRecognition instance (window.SpeechRecognition || window.webkitSpeechRecognition). Sets continuous to false, interimResults to false, maxAlternatives to 1, lang to "en-IN". Calls start(). Sets a 15-second timeout that calls stop() if no result fires. On result event, takes event.results[0][0].transcript. Returns the transcript string.

Audio capture — manual fallback mode. If SpeechRecognition is not available or if the first capture attempt fails with a network error (indicating the device requires connectivity for recognition), switch to manual mode. Show a text input field labeled "Type what you heard." The user types the content. This produces the same output as speech recognition — a transcript string — and proceeds identically through the rest of the pipeline.

Audio message creation from transcript. After receiving a transcript string (from either mode), show it to the user in an editable text area. User can correct obvious transcription errors. User selects a priority (default 3). User taps Save. The app creates a message with type "audio", confidence "low", payload set to the edited transcript, and runs Ingestion. Returns the created message object.

Capture session UI states. The UI has four states: idle (show Listen button), listening (show waveform animation and Stop button), review (show editable transcript and Save/Discard buttons), saved (brief confirmation then back to idle).

Phase 5 tests:

Test 5.1 — Call broadcastMessage() with a text message. Verify Speech Synthesis fires without errors. Verify rate is 0.9 and volume is 1.0.

Test 5.2 — Call broadcastMessage() with an alert type message. Verify the spoken text begins with "URGENT ALERT: ".

Test 5.3 — Queue two messages for broadcast with one alert in second position. Verify the alert is spoken first despite being queued second.

Test 5.4 — On a device with on-device speech recognition (iOS preferred), start capture and speak a short sentence clearly. Verify a non-empty transcript is returned within 15 seconds.

Test 5.5 — Start capture on a device where speech recognition requires network, with network disabled. Verify the function detects the network error and switches to manual fallback mode. Verify the text input UI appears.

Test 5.6 — Complete a full capture → review → edit → save flow. Verify the saved message in storage has type "audio", confidence "low", and payload matching the edited transcript text.

Test 5.7 — Save an audio message. Verify hop_count is 0 (it was created on this device). Verify weight is 1.0 (not yet relayed). Verify it appears in the message list with a visual "low confidence" indicator.

Test 5.8 — Audio message relay test. Device A creates an audio message via capture. Device A syncs with Device B via QR or WebRTC. Device B receives the audio message. Verify type is "audio" and confidence is "low" on Device B. Verify Device B can broadcast it via Speech Synthesis.

Test 5.9 — Broadcast a message on Device A. Have a human listener on Device B activate capture. Verify Device B captures a transcript that approximates the original text. This is a human-in-the-loop test — note that perfect transcription is not expected and the pass condition is that the flow completes end-to-end without errors, not that the transcript is perfect.

FRONTEND CONTRACT

These are the exact functions the UI calls and the exact event shapes it listens to. The frontend can be built against this spec independently.

Functions exposed by backend:

createMessage(type, priority, payload, ttl_hours) → Promise resolving to message object or throwing with reason string.

getAllMessages() → Promise resolving to array of message objects with computed fields, sorted by score descending.

getStorageStats() → Promise resolving to { used_bytes, message_count, hot_count, warm_count, cold_count, quarantine_count }.

generateSnapshotQR() → Promise resolving to base64 PNG data URL string.

generateWebRTCOfferQR() → Promise resolving to base64 PNG data URL string. Also begins WebRTC offer process internally.

startScan() → Promise resolving to { mode: "snapshot", results: [ingest_result...] } or { mode: "webrtc_answer", sdp: string } or { mode: "unknown", raw: string }.

processWebRTCAnswer(sdp_string) → Promise resolving to void when data channel opens.

startJoinerFlow(offer_sdp) → Promise resolving to base64 PNG data URL of answer QR.

processInitiatorAnswer(answer_sdp) → Promise resolving to void when data channel opens.

broadcastMessage(message_id) → Promise resolving to void when speech ends.

startCapture() → Promise resolving to transcript string, or opening manual fallback UI.

saveAudioMessage(transcript, priority) → Promise resolving to message object.

setPIN(pin_string) → Promise resolving to void on success.

verifyPIN(pin_string) → Promise resolving to true or false. Increments internal wrong-attempt counter. Triggers Level 2 wipe on third consecutive false.

clearPIN() → Promise resolving to void. Requires current PIN to be verified first (call verifyPIN before clearPIN).

triggerManualWipe() → void (synchronous, no return — page reloads).

Events emitted by backend (via a simple EventEmitter or CustomEvent on window):

messages_updated — fired after every scoring loop. No payload. UI should call getAllMessages() on receipt.

ingest_complete — fired after successful ingest. Payload: { message_id, status }.

sync_progress — fired during WebRTC session. Payload: { sent, received, peer_message_count, direction: "sending" | "receiving" | "both" }.

sync_complete — fired when both peers send SYNC_DONE. Payload: { total_sent, total_received, session_id }.

sync_error — fired on data channel error or unexpected close. Payload: { reason }.

storage_warning — fired when storage exceeds 40MB. No payload. UI shows storage warning.

eviction_complete — fired after eviction runs. Payload: { deleted_count, remaining_count }.

sentinel_triggered — fired when triple shake detected, before wipe runs. UI has approximately 200ms to respond (show a flash or nothing — wipe proceeds regardless).

capture_mode_changed — fired when audio capture falls back to manual mode. Payload: { reason: "api_unavailable" | "network_required" }.

ttl_sweep_complete — fired after expiry sweep. Payload: { expired_count }.

DEMO SCRIPT — PHASES 2 AND 3 COMBINED

This is the exact flow to demonstrate A→B→C with QR and WebRTC.

Setup before demo:

- Three devices, all in airplane mode with Wi-Fi re-enabled
- All three have the PWA installed and loaded
- Device B creates a Wi-Fi hotspot (other devices join it for WebRTC local network connectivity — WebRTC needs a shared network but no internet)
- Devices A and C join Device B's hotspot

Demo flow:

Step 1: Device A creates a text message: "Northern route is clear, avoid central market." Priority 4.

Step 2: Device A generates Snapshot QR.

Step 3: Device B scans Device A's QR. Message appears on B with hop_count 1. (QR transfer demonstrated.)

Step 4: Device B taps "Connect via WebRTC" — generates offer QR.

Step 5: Device C scans Device B's offer QR. Device C generates answer QR.

Step 6: Device B scans Device C's answer QR. Data channel opens. Sync session runs. Message appears on C with hop_count 2. (WebRTC live sync demonstrated.)

Step 7: Show all three screens. Same message, hop_count 0 on A (creator), 1 on B, 2 on C.

Step 8: Device A creates an alert: "Checkpoint established at south gate." Priority 5.

Step 9: Wait 15 seconds. Message propagates from A to B via a new QR or a second WebRTC session. From B to C via the still-open or re-established channel. Show alert appearing on C with a different visual treatment.

Step 10 (Sentinel demo, optional): Ask a judge to attempt to seize Device B. Operator triple-shakes Device B. Screen goes blank. Database is wiped. Message that was on B is gone. Devices A and C are unaffected — they have their own copies.

Total demo time: approximately 4 minutes including setup. Trim to 2 minutes by preparing step 1-3 before judges approach.

LIBRARY VERSIONS AND BUNDLE

All libraries bundled at build time. No CDN calls at runtime.

idb: 7.x — IndexedDB wrapper
pako: 2.x — zlib compression/decompression
qrcode: 1.5.x — QR code generation, browser build
jsQR: 1.4.x — QR code decoding from ImageData
bcryptjs: 2.4.x — PIN hashing (pure JS, no native dependencies)

WebRTC, SpeechSynthesis, SpeechRecognition, DeviceMotionEvent — all native browser APIs, no libraries.

Total bundle size estimate: approximately 180KB before gzip. Well within PWA constraints and loads instantly from Service Worker cache.

WHAT IS EXPLICITLY OUT OF SCOPE

Network Cartographer — not built, described in presentation as future work.

Physical QR drop points — not needed for hackathon demo.

Encounter classification (full sync vs burst) — not built, WebRTC sessions are always full sync.

Message fragmentation across devices — not built, privacy-through-no-identity plus Sentinel wipe covers the requirement.

Data poisoning on wipe — not built, clean wipe only.

Animated multi-frame QR — not built, compression fits messages in single QR.

Any server infrastructure — explicitly prohibited by the problem statement and by the PWA decision.